Programación Orientada a Servicios

Service-Oriented Programming

Universidad de Montemorelos
Facultad de Ingeniería y Tecnología (FITEC)
Ingeniería en Sistemas Computacionales

Agenda de la Clase

01

¿Qué es SOA?

Definición y fundamentos de la Arquitectura Orientada a Servicios

02

Principios de Diseño

Contrato estandarizado, bajo acoplamiento, abstracción, reutilización

03

Servicios Web

REST, SOAP, y protocolos de comunicación entre servicios

04

SOA vs Microservicios

Diferencias, ventajas y cuándo usar cada enfoque

05

Ejercicio Práctico

Sistema hospitalario con Spring Boot — servicios reutilizables

¿Qué es la Arquitectura Orientada a Servicios?

SOA (Service-Oriented Architecture)

SOA es un método de desarrollo de software que estructura las aplicaciones como una colección de servicios independientes que se comunican entre sí.

Componentes modulares

Cada servicio encapsula una capacidad de negocio específica

Interoperabilidad

Los servicios se comunican mediante protocolos estándar (HTTP, SOAP, REST)

Reutilización

Un servicio puede ser consumido por múltiples aplicaciones

Composición

Servicios simples se combinan para construir procesos complejos

Ejemplo: Sistema Hospitalario

En una organización de atención médica, múltiples sistemas necesitan funcionalidades comunes. En lugar de duplicar código, se crean servicios reutilizables.

Servicio de Autenticación

Servicio de Registro Pacientes

Servicio de Notificaciones

Sistemas que consumen los servicios:

Sistema de Administración

Usa: Autenticación, Notificaciones

Portal del Paciente

Usa: Autenticación, Registro, Notificaciones

Sistema EHR (Historial Electrónico)

Usa: Autenticación, Registro

Sistema de Facturación

Usa: Autenticación, Registro, Notificaciones

Principios de Diseño SOA

Contrato Estandarizado

Los servicios definen interfaces claras y documentadas (APIs)

Bajo Acoplamiento

Los servicios minimizan dependencias entre sí

Abstracción

Ocultan su lógica interna, exponen solo lo necesario

Reutilización

Diseñados para ser usados por múltiples consumidores

Autonomía

Cada servicio controla su propia lógica y recursos

Sin Estado (Stateless)

No retienen información de sesiones previas

Servicios Web: REST vs SOAP

REST

- Usa HTTP estándar (GET, POST, PUT, DELETE)
- Formato ligero: JSON (o XML)
- Sin estado (stateless) por diseño
- Fácil de consumir desde cualquier cliente
- Ideal para aplicaciones web y móviles
- Spring Boot lo implementa con @RestController

SOAP

- Protocolo XML estricto con WSDL
- Soporta WS-Security (más seguro)
- Transacciones ACID confiables
- Mayor overhead por mensajes XML
- Común en sistemas bancarios y legacy
- Más complejo pero más robusto

En este curso usaremos REST con Spring Boot — el estándar de la industria actual para SOA moderna.

SOA vs Microservicios

Aspecto	SOA	Microservicios
Alcance	Nivel empresarial	Nivel de aplicación
Comunicación	ESB (Enterprise Service Bus)	API Gateway / HTTP directo
Tamaño del servicio	Servicios más grandes y completos	Servicios muy pequeños y específicos
Base de datos	Compartida entre servicios	Cada servicio tiene la suya
Despliegue	Monolítico o coordinado	Independiente por servicio
Gobernanza	Centralizada	Descentralizada
Tecnología	Homogénea (un stack)	Políglota (diferentes stacks)

Ejercicio Práctico

Sistema de Gestión Hospitalaria con Spring Boot

Implementando servicios reutilizables en una arquitectura SOA real

Arquitectura del Ejercicio

Crearemos 3 microservicios REST que se comunican entre sí, simulando un sistema hospitalario real.

auth-service

Puerto: 8081

Login, JWT Tokens,
Validación de credenciales

patient-service

Puerto: 8082

Registro de pacientes,
CRUD de expedientes

notification-service

Puerto: 8083

Envío de emails,
Alertas del sistema

hospital-gateway (Puerto 8080)

API Gateway que orquesta los 3 servicios — consume auth-service, patient-service y notification-service usando RestTemplate / WebClient

Paso 1: auth-service

Servicio de autenticación reutilizable

AuthController.java

```java
@RestController
@RequestMapping("/api/auth")
public class AuthController {
    @Autowired
    private AuthService authService;

    @PostMapping("/login")
    public ResponseEntity<AuthResponse> login(
            @RequestBody LoginRequest request) {
        AuthResponse response = authService
                .authenticate(request);
        return ResponseEntity.ok(response);
    }

    @GetMapping("/validate/{token}")
    public ResponseEntity<Boolean> validateToken(
            @PathVariable String token) {
        boolean isValid = authService.validate(token);
        return ResponseEntity.ok(isValid);
    }
}
```

Paso 2: patient-service

Servicio de registro de pacientes — reutilizado por todos los sistemas

PatientController.java

```java
@RestController
@RequestMapping("/api/patients")
public class PatientController {
    @Autowired
    private PatientService patientService;

    @PostMapping("/register")
    public ResponseEntity<Patient> register(
            @RequestBody PatientDTO dto) {
        Patient patient = patientService.register(dto);
        return ResponseEntity
                .status(HttpStatus.CREATED).body(patient);
    }

    @GetMapping("/{id}")
    public ResponseEntity<Patient> getById(
            @PathVariable Long id) {
        return ResponseEntity
                .ok(patientService.findById(id));
    }

    @GetMapping
    public ResponseEntity<List<Patient>> getAll() {
        return ResponseEntity.ok(patientService.findAll());
    }
}
```

Paso 3: hospital-gateway

El gateway consume los servicios — principio SOA de composición

HospitalGatewayService.java

```java
@Service
public class HospitalGatewayService {
    @Autowired
    private RestTemplate restTemplate;
    private static final String AUTH_URL =
            "http://localhost:8081/api/auth";
    private static final String PATIENT_URL =
            "http://localhost:8082/api/patients";

    // Reutiliza el servicio de autenticación
    public AuthResponse login(LoginRequest req) {
        return restTemplate.postForObject(
            AUTH_URL + "/login", req, AuthResponse.class);
    }
    // Reutiliza el servicio de registro
    public Patient registerPatient(PatientDTO dto) {
        return restTemplate.postForObject(
            PATIENT_URL + "/register", dto, Patient.class);
    }
    // Composición: autentica y luego registra
    public Patient secureRegister(
            LoginRequest auth, PatientDTO patient) {
        AuthResponse token = login(auth);
        return registerPatient(patient);
    }
}
```

Instrucciones del Ejercicio

1

Crear 3 proyectos Spring Boot

auth-service (8081), patient-service (8082), notification-service (8083). Cada uno independiente con su propio application.properties.

2

Implementar los endpoints REST

Cada servicio expone sus endpoints con @RestController. Usar DTOs para transferencia de datos entre servicios.

3

Crear el Gateway (puerto 8080)

Usar RestTemplate o WebClient para consumir los 3 servicios. Implementar al menos 2 endpoints que combinen servicios (composición).

4

Probar con Postman o cURL

Verificar que el gateway puede autenticar un usuario y luego registrar un paciente en una sola operación compuesta.

5

Entregable

Subir los 4 proyectos a un repositorio Git. Incluir un README.md con instrucciones de ejecución y capturas de las pruebas.

Resumen

- SOA estructura el software como servicios independientes y reutilizables
- Los principios clave son: bajo acoplamiento, abstracción, reutilización y composición
- REST es el protocolo predominante en SOA moderna (Spring Boot + @RestController)
- Los servicios se comunican via HTTP — consumidos con RestTemplate o WebClient
- La composición de servicios permite construir flujos empresariales complejos

"Educar es redimir" — Universidad de Montemorelos